entities. In fact, JPA and Spring Framework on the Java platform offer two popular annotations @Entity and @Service to clearly mark the classes. And this practice is working. What is wrong with it?

Neither the Spring Framework nor the annotations are wrong. It is the practice of thinking from the perspective of database design that is wrong. Indirectly, we are designing the objects depending on the database technology that we choose. We freeze the database architecture, the schema, and the queries early in the design. The problem with this approach is that the rest of the system is designed at the mercy of what is offered by the database layer. Everything is retrofitted into the frozen database layer. It makes the system too rigid for a change. And, as you know, any system that is not agile enough to absorb the change obsoletes quickly.

The buck does not stop even there. The project teams often choose a platform, besides the database, before designing the objects in the business layer. For instance, the way a Python developer designs the objects is different from the way a Java developer designs them. Another disturbing observation is that even on a single platform like Java, the objects designed for the Jersey framework look completely different from the objects designed for the Spring Framework. Objects are supposed to reflect real-world entities, isn't it! However, in practice, it is the choice of database, platform, or frameworks that is driving the design. This is wrong because a rigid monolith system cannot respond to changes in business requirements. The DDD attempts to put the domain in the front seat, not the technology.

This looks reasonable. However, most of the people who are involved in software development are more technology experts, not domain experts. Isn't this an obstacle to adopting DDD?

This is one of the anomalies in the industry. The teams that gather the system requirements are often ignorant of the technology, and the teams that design and build the system solutions are ignorant of the business domain. It is a common practice for a young engineer to jump from one team to another for career growth. This often results in jumping from one domain to another domain itself. It is no surprise to see a designer moving from the financial domain to the e-commerce domain. Though this gives exposure to various domains, the person will not gain sufficient expertise in any of the domains. Ultimately, such a designer gets carried away by the technology he or she knows, rather than by the nuances of the actual domain to which the system belongs.

The DDD strongly vouches for domain expertise. The overall design of the system, granular design of the objects, and their interactions should reflect the corresponding domain. How the teams gain such expertise is left to the individuals and the organisations.

Still, it is inevitable to have separate teams for product marketing and engineering. Does DDD have any solution to iron out the communication gap between the teams?

The DDD suggests that the engineering team should refrain from technical jargon. Instead, the teams are advised to develop a consistent vocabulary with a strong affinity to the domain. The vocabulary is referred to as ubiquitous language. All the stakeholders of the team are expected to use the same language. Without such a language, a specific word means many things to many people.

For instance, an e-commerce system consists of a *subscriber* object, a database table named *customer*, and a label on the user interface that reads as the *user*. Though these look as if they are three different entities from the outset, they may all actually refer to just one entity. Instead, all the stakeholders should stick only to one word. And that word should come from the business. If that specific e-commerce company refers to them as *customers*, then the word *customer* should be chosen, nothing else. It is even better to maintain a dictionary of the vocabulary.

In summary, DDD advises modelling the objects that correctly map to the actual business domain and using the language that the business uses. In a way, it fits into the Onion Model. Isn't it?

The Onion Model consists of a domain layer, service layer, application layer, and infrastructure layer. The domain layer comprises technology-agnostic objects. The design of this layer is purely driven by the domain, nothing else. A domain object consists of both *state* and *behaviour*. However, there are places where more than one domain object needs to be involved in a business workflow. The service layer consists of objects that deal with more than one such domain object. The application layer acts as the gateway to the rest of the world. This is the layer where requests and responses are handled. The infrastructure layer is largely specific to the technology. It consists of databases, messaging systems, test environment, user interface, etc. It is natural that the DDD approach and the Onion Model go hand-in-hand.

However, DDD is not just limited to it. The DDD strongly advises decomposing a monolith domain into smaller sub-domains, and to deal with each of the sub-domains independently. In other words, the Onion Model is applied for each of the sub-domains.

If the sub-domains are handled independently, how are the domain objects that belong to more than one sub-domain handled? Are the objects duplicated or is the effort duplicated?